

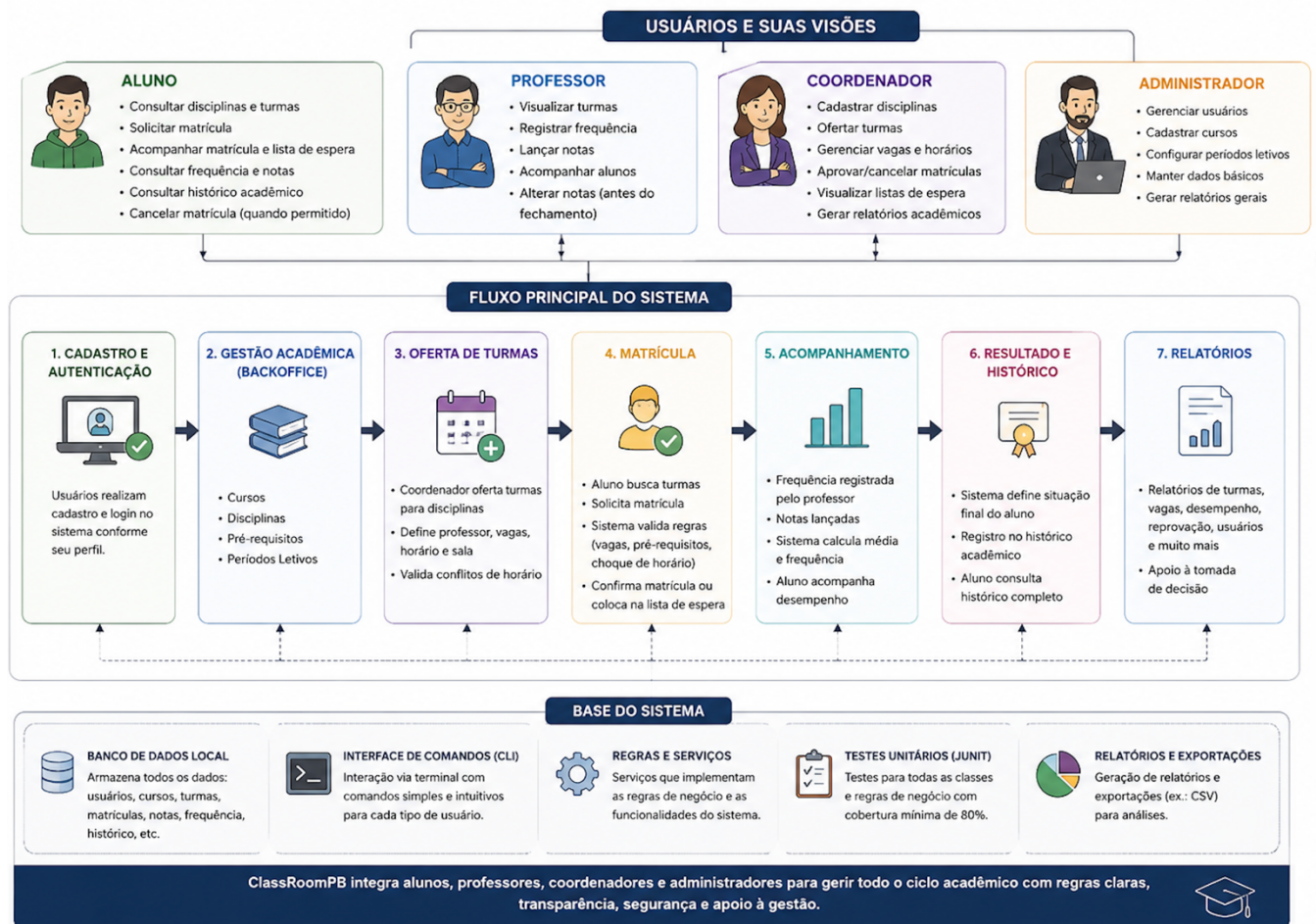
PROJETO – Eng.Sw.2 2026.1

- Este projeto deve ser feito em equipe 3 a 4 alunos.
- Cada grupo deverá criar um repositório no Github para acompanhamento.
 - Pelo repositório será possível acompanhar as contribuições de cada membro do grupo. Se vocês fizerem programação em pares, é importante dividir os commits entre os dois membros alternadamente (um dia faz o commit no nome de um dos alunos e no outro dia faz o commit no nome do outro aluno para termos certeza que todos estão trabalhando).

• SISTEMA: ClassroomPB

Se trata de um sistema de *controle acadêmico simplificado*, o CassRoonPB.

Onde existem basicamente 4 tipos de usuário (cada um com uma visão diferente do sistema): aluno, professor, coordenador e administrador.



- **Requisitos Gerais**

A seguir temos uma breve descrição requisitos funcionais desejados para o sistema (o que o sistema deve fazer). Observe que, no mundo real, o cliente poderá mudar de idéia a respeito desses requisitos funcionais ao longo do tempo. É até possível que a professora altere os requisitos no meio do semestre. Os requisitos foram priorizados pelo cliente conforme podem ver na descrição das releases mais adiante. Não iremos tratar de requisitos não-funcionais.

Cadastro e Autenticação

RF01: O sistema deve permitir o cadastro de alunos, professores, coordenadores e administradores.

RF02: O sistema deve permitir login com matrícula/e-mail e senha.

RF03: O sistema deve validar perfis de acesso, exibindo funcionalidades diferentes para cada tipo de usuário.

RF04: O sistema deve impedir cadastro duplicado por matrícula ou e-mail.

Gestão Acadêmica Básica

RF05: O administrador deve poder cadastrar cursos.

RF06: O coordenador deve poder cadastrar disciplinas.

RF07: Cada disciplina deve possuir código, nome, carga horária, créditos e pré-requisitos opcionais.

RF08: O coordenador deve poder cadastrar períodos letivos, como “2026.2”.

RF09: O sistema deve permitir ativar ou encerrar um período letivo.

Oferta de Turmas

RF10: O coordenador deve poder ofertar turmas para uma disciplina em um período letivo.

RF11: Cada turma deve possuir professor responsável, limite de vagas, horário e sala.

RF12: O sistema deve impedir choque de horário para o professor.

RF13: O sistema deve impedir oferta de turma sem professor responsável.

RF14: O coordenador deve poder alterar ou cancelar uma turma antes do início das aulas.

Matrícula

RF15: O aluno deve poder consultar disciplinas/turmas disponíveis.

RF16: O aluno deve poder solicitar matrícula em uma turma.

RF17: O sistema deve verificar se há vagas disponíveis.

RF18: O sistema deve verificar se o aluno cumpriu os pré-requisitos da disciplina.

RF19: O sistema deve impedir choque de horário entre turmas do mesmo aluno.

RF20: O sistema deve confirmar matrícula automaticamente quando todos os critérios forem atendidos.

RF21: Caso não haja vaga, o aluno deve entrar em lista de espera.

RF22: O aluno deve poder cancelar matrícula dentro do período permitido.

Lista de Espera

RF23: O sistema deve manter lista de espera por turma.

RF24: Quando uma vaga for liberada, o sistema deve chamar automaticamente o próximo aluno da lista.

RF25: A lista de espera deve respeitar a ordem de solicitação.

RF26: O coordenador deve poder visualizar a lista de espera de cada turma.

Frequência

RF27: O professor deve poder registrar presença/falta dos alunos por aula.
RF28: O sistema deve calcular automaticamente o percentual de frequência.
RF29: O aluno deve poder consultar sua frequência por disciplina.
RF30: O sistema deve alertar quando o aluno estiver abaixo do mínimo exigido.

Notas e Avaliação

RF31: O professor deve poder lançar notas das etapas (etapa1 e etapa2).
RF32: O sistema deve calcular automaticamente a média final.
RF33: O aluno deve poder consultar suas notas.
RF34: O sistema deve informar a situação do aluno: aprovado, reprovado por nota, reprovado por falta ou em recuperação.
RF35: O professor deve poder alterar notas antes do fechamento da turma.

Histórico Acadêmico

RF36: O sistema deve manter histórico das disciplinas cursadas pelo aluno.
RF37: O histórico deve registrar período, disciplina, professor, nota final, frequência e situação.
RF38: O aluno deve poder consultar seu histórico acadêmico.
RF39: O coordenador deve poder consultar o histórico dos alunos do curso.

Relatórios

RF40: O coordenador deve gerar relatório de alunos matriculados por turma.
RF41: O coordenador deve gerar relatório de ocupação de vagas.
RF42: O coordenador deve gerar relatório de reprovação por disciplina.
RF43: O administrador deve gerar relatório geral de usuários cadastrados.

RF_SurpresaN: Requisitos surpresa a ser definido durante o semestre.

Regras de Negócio Importantes

- RN01: Um aluno não pode se matricular duas vezes na mesma turma.
- RN02: Um aluno não pode se matricular em turmas com horários conflitantes.
- RN03: Uma turma não pode ultrapassar o número máximo de vagas.
- RN04: Uma disciplina pode ter pré-requisitos.
- RN05: Um aluno só pode cursar disciplina se tiver sido aprovado nos pré-requisitos.
- RN06: Professor não pode ministrar duas turmas no mesmo horário.
- RN07: Notas devem estar entre 0 e 10.
- RN08: Frequência mínima para aprovação: 75%.
- RN09: Média mínima para aprovação direta: 7,0.
- RN10: Média entre 4,0 e 6,9 gera recuperação.
- RN11: Média abaixo de 4,0 reprova por nota.
- RN12: Reprovação por falta prevalece sobre aprovação por nota.

• CONSIDERAÇÕES:

- Testes: devem ser feitos testes para cada classe com o JUnit e ser calculada a cobertura (mínimo de 80%). Para o Eclipse tem o plugin Emma.
- Tratamento de erros: deve ser feito tratamento de erros para cada requisito, por exemplo: não deve ser possível cadastrar um usuário que já existe na plataforma, ou modificar um usuário removido etc. Pensem em todas as possibilidades de erro e deem o devido tratamento.

- **Persistência:** o armazenamento deve ser local, ou seja, o sistema deve armazenar em disco todos os dados necessários para manter o seu estado mesmo que o programa seja fechado. Assim, se eu usar o programa, adicionar usuários, corridas, etc. tudo isso deve ser visto se eu fechar o programa e abrir novamente.
- **Interface:** vocês vão observar que não pedi interface gráfica, motivo: não quero que você gaste tempo com interfaces gráficas, pois temos outras disciplinas para isso, o foco deve ser na qualidade do projeto. Porém, deve ser feita uma interface de comandos simplificada.
- **Não é para usar API (por ex. Swagger e afins), usar apenas Java, JUnit e padrões de Projeto!**

Exemplo de interface:

```

cadastrarAluno nome cpf matricula email senha
login email senha
cadastrarDisciplina codigo nome cargaHoraria creditos
ofertarTurma codigoDisciplina professor periodo vagas horario sala
solicitarMatricula matriculaAluno codigoTurma
cancelarMatricula matriculaAluno codigoTurma
registrarFrequencia codigoTurma data matriculaAluno presente
lançarNota codigoTurma matriculaAluno avaliacao nota
consultarHistorico matriculaAluno
gerarRelatorioOcupacao

```

• **SOBRE ENTREGAS:**

- Cada release deve conter:
 - **Repositório** contendo a implementação dos requisitos relacionado àquela entrega, juntamente com a parte de persistência, a interface simplificada, os testes e o tratamento de erros referentes aos requisitos daquela entrega;
 - Nesse repositório, além do código, crie um pasta chamada “*releases*”, lá você deve adicionar o relatório dessa entrega (chamado **relatorio-release1.pdf**), que deve conter:
 - (1) A descrição da arquitetura utilizada (diagramas de caso de uso, de classes e de sequência)
 - (2) As funcionalidades desenvolvidas e prints do terminal com exemplos das funcionalidades
 - (3) Relatório de testes e cobertura de código
 - Nesse repositório, ainda na pasta “*releases*”, adicione o relatório de gerenciamento do processo de desenvolvimento dessa entrega (chamado **relatorio-processo-release1.pdf**), que deve conter, para cada sprint:
 - (1) O backlog com os itens de trabalho e as tasks;
 - (2) A descrição da organização da equipe, os papéis de cada membro;
 - (3) Gráfico de burndown de cada sprint, o Azure gera esse gráfico, basta fazer um print e colocar no relatório.

• **AVALIAÇÃO**

- O projeto será avaliado de acordo com os seguintes pesos:
 - Compilação (0% - se não compilar, o projeto não deve ser aceito)

- Repositório + apresentação das funcionalidades + testes (30%)
- Relatório de release (20%)
- Relatório de gerenciamento do processo de desenvolvimento (50%)

• ENTREGAS

- **Release1** – Núcleo Acadêmico
Objetivo: criar a base do sistema.
Requisitos: RF01 a RF14
Entregas: cadastro/login, cursos, disciplinas, períodos letivos, turmas, professores, validações básicas, persistência e testes.
- **Release2** – Matrícula e Acompanhamento
Objetivo: implementar o fluxo principal do aluno.
Requisitos: RF15 a RF30
Entregas: consulta de turmas, matrícula, lista de espera, cancelamento, frequência, cálculo de percentual, tratamento de erros, persistência e testes.
- **Release3** – Avaliação, Histórico e Evolução
Objetivo: fechar o ciclo acadêmico.
Requisitos: RF31 a RF45
Entregas: lançamento de notas, cálculo de média, situação final, histórico acadêmico, relatórios, tratamento de erros, persistência e testes.
- **Release4** – Troca de projetos
 - Novos requisitos

• CRONOGRAMA DE SPRINTS

- SPRINT1:
- SPRINT2:
- SPRINT3:
- SPRINT4:
- SPRINT5:
- SPRINTN: